SYSTEMS ANALYSIS AND DESIGN

Classes & Objects in Python

Part 2 · Designing with Objects

Dr. Pejman Ebrahimi

Today’s roadmap

From single classes to classes that work together



01

Objects working together

Objects inside objects, None



02

Encapsulation

Protecting data, @property, errors



03

Class-level data

Class variables, class & static methods



04

Inheritance & polymorphism

Reusing and extending classes

Warm-up: what does this print?

A 3-minute recap of Part 1 — discuss with your neighbour

recap.py

```
class Cup:
    def __init__(self, size):
        self.size = size
        self.filled = 0

    def pour(self, amount):
        self.filled += amount
        if self.filled > self.size:
            self.filled = self.size

a = Cup(300)
b = Cup(200)
a.pour(250)
b.pour(500)
print(a.filled, b.filled)
```

Questions

1. Which line is the **constructor**?
2. Name the two **attributes**.
3. How many **objects** are created?
4. What is printed at the end?

Warm-up: answers

recap.py

```
class Cup:
    def __init__(self, size):
        self.size = size
        self.filled = 0

    def pour(self, amount):
        self.filled += amount
        if self.filled > self.size:
            self.filled = self.size

a = Cup(300)
b = Cup(200)
a.pour(250)
b.pour(500)
print(a.filled, b.filled)
```

Answers

1. `def __init__(self, size):`
2. `size` and `filled`
3. Two: `a` and `b`
4. The cup cannot overflow:

Output

```
250 200
```

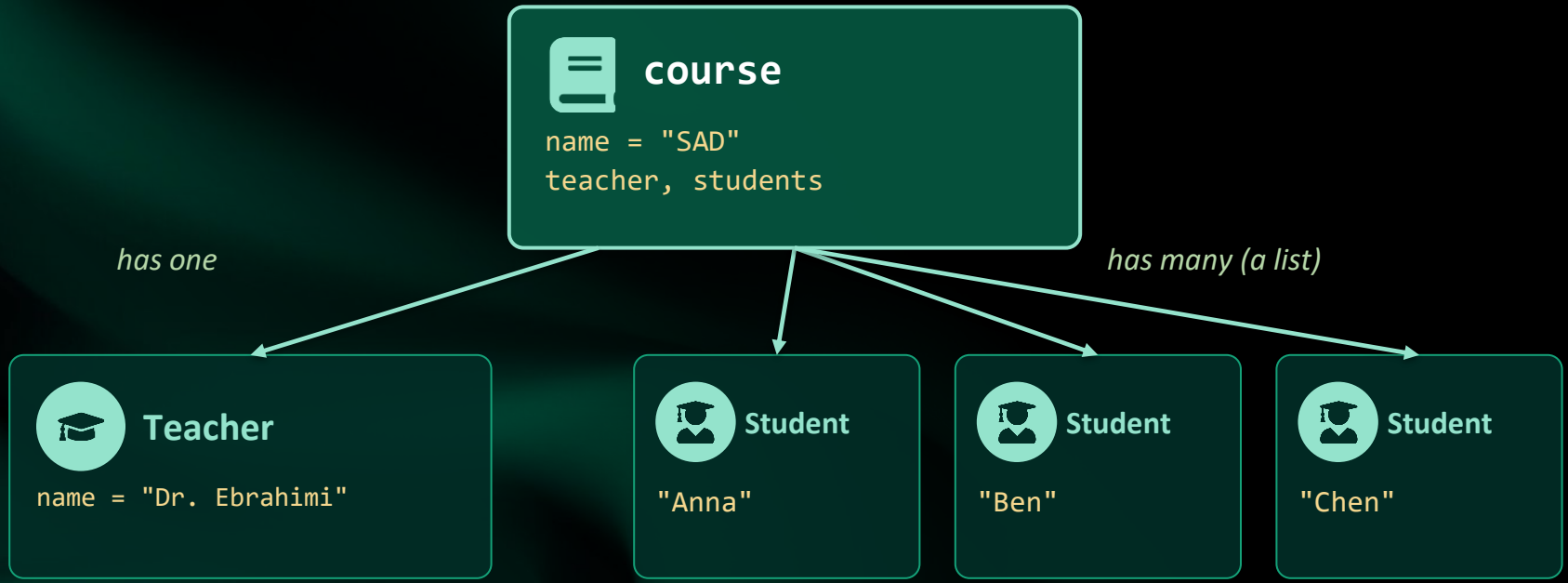
01

Objects working together

Objects inside objects — the has-a relationship

Objects can contain other objects

Real systems are made of objects that are connected



This is called **composition** or a **has-a** relationship: a course has a teacher and has students.

An object as an attribute

Follow the dots: course → teacher → name

course.py

```
class Teacher:
    def __init__(self, name, email):
        self.name = name
        self.email = email

class Course:
    def __init__(self, name, teacher):
        self.name = name
        self.teacher = teacher  # Teacher object!

t = Teacher("Dr. Ebrahimi", "pejman.ebrahimi@uni.li")
sad = Course("SAD", t)
print(sad.teacher.name)
print(sad.teacher.email)
```

Output

```
Dr. Ebrahimi
pejman.ebrahimi@uni.li
```



Read it left to right

`sad.teacher` is a Teacher object, so `sad.teacher.name` asks that teacher for its name.

A list of objects as an attribute

The course manages its own list of students

course.py

```python
class Student:
    def __init__(self, name, grade):
        self.name = name
        self.grade = grade

class Course:
    def __init__(self, name):
        self.name = name
        self.students = []          # starts empty

    def enroll(self, student):
        self.students.append(student)

    def average_grade(self):
        total = sum(s.grade for s in self.students)
        return total / len(self.students)

sad = Course("SAD")
sad.enroll(Student("Anna", 5.5))
sad.enroll(Student("Ben", 4.5))
print(len(sad.students), sad.average_grade())
```

Output

```
2 5.0
```



Good design

Other code does not touch `students` directly.

It asks the course: `enroll()`, `average_grade()`.

In Switzerland & Liechtenstein, 6 is the best grade.

None: a reference to nothing

Use it when there is no object to return

none_demo.py

```python
# Student class: see previous slide
def find_student(students, name):
    for s in students:
        if s.name == name:
            return s
    return None              # nobody found

group = [Student("Anna", 5.5),
         Student("Ben", 4.5)]
result = find_student(group, "Zoe")

if result is None:
    print("No such student")
else:
    print(result.grade)
```

Output

```
No such student
```



Watch out

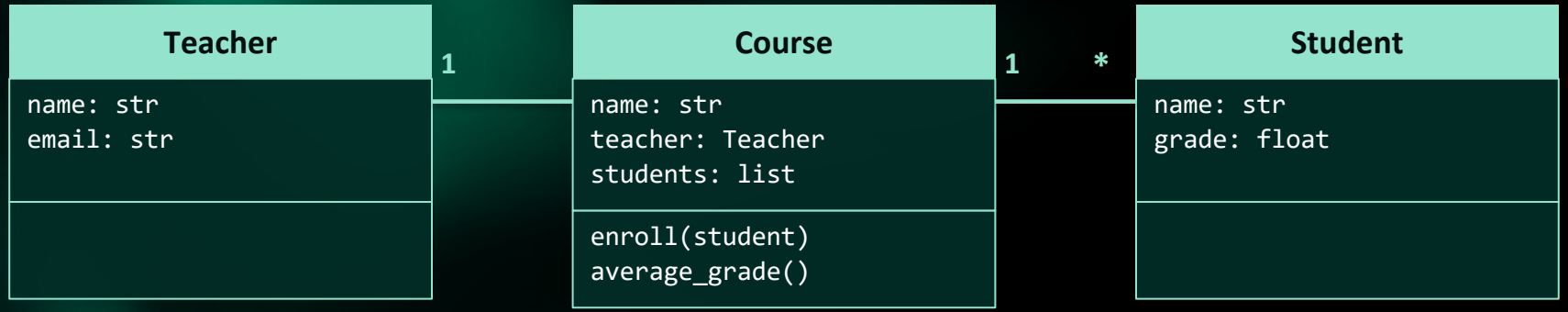
Calling `result.grade` when `result` is `None` crashes:

AttributeError: 'NoneType' object has no attribute 'grade'

Always check with `is None` first.

Modelling has-a relationships in UML

Lines between classes show which objects know each other



Multiplicity

1 = exactly one, * = many (zero or more). A course has **1** teacher and **many** students.

In Python

“1” becomes a single object attribute; “many” becomes a **list** of objects.

In-class task 1: Team and Player

15 min

Work in pairs · 15 minutes



Your task

- `Player` with `name` and `goals`
- `Team` with a `name` and a list `players` (starts empty)
- `add_player(player)`
- `total_goals()` returns the sum of all goals
- `top_scorer()` returns the Player with most goals, or `None` if the team is empty
- Draw the UML diagram first!

test it like this

```
t = Team("FC Vaduz")
print(t.top_scorer())
t.add_player(Player("Lea", 7))
t.add_player(Player("Max", 12))
print(t.total_goals())
print(t.top_scorer().name)
```

Expected output

```
None
19
Max
```

02

Encapsulation

Protect the data, offer safe methods

The problem: anyone can break the rules

Our withdraw() method checks the balance — but nothing forces people to use it

broken.py

```
# BankAccount class from Part 1
acc = BankAccount("Anna", 100)

acc.withdraw(500)    # refused
print(acc.balance)

acc.balance = -5000  # allowed!
print(acc.balance)
```

Output

```
100
-5000
```



Why is that bad?

- The object can end up in an **invalid state**
- Bugs appear far away from where the mistake was made
- In a big system, many developers use your class — you cannot trust everyone to be careful

Encapsulation: hide the data, offer methods

Like a cash machine: you never touch the money directly



public methods = the only doors

Data is **hidden** inside.

All access goes through methods that check the rules.

Public, protected and private attributes

Python uses naming conventions with underscores



`name`

Public
Anyone may read and change it



`_name`

Protected
Convention: “internal — please do not touch from outside”



`__name`

Private
Python hides it: access from outside raises an error

private.py

```python
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.__balance = balance # private

acc = BankAccount("Anna", 100)
print(acc.owner)
print(acc.__balance)
```

Output

```
Anna
Traceback (most recent call last):
AttributeError: 'BankAccount' object has no attribute '__balance'
```

Getters and safe methods

Read with a getter, change only through methods that check the rules

bank_account.py

```python
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.__balance = balance

    def get_balance(self):          # getter
        return self.__balance

    def deposit(self, amount):      # safe method
        if amount <= 0:
            print("Amount must be positive")
            return
        self.__balance += amount

    def withdraw(self, amount):     # safe method
        if amount > self.__balance:
            print("Not enough money")
            return
        self.__balance -= amount
```

usage

```python
acc = BankAccount("Anna", 100)
acc.deposit(-20)
acc.withdraw(500)
acc.deposit(50)
print(acc.get_balance())
```

Output

```
Amount must be positive
Not enough money
150
```

The Pythonic way: @property

Looks like a normal attribute from outside, but runs your checks

product.py

```python
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price        # uses setter!

    @property
    def price(self):              # getter
        return self.__price

    @price.setter
    def price(self, value):       # setter
        if value < 0:
            raise ValueError("Negative price")
        self.__price = value
```

usage

```python
p = Product("Laptop", 999)
print(p.price)
p.price = 899
print(p.price)
p.price = -1
```

Output

```
999
899
Traceback (most recent call last):
ValueError: Negative price
```

Raising and catching errors

raise stops the program with a clear message; try/except handles it

raise: signal a problem

```
def set_age(self, age):
    if age < 0 or age > 130:
        raise ValueError("Invalid age")
    self.__age = age
```

try / except: handle it

```
try:
    p = Product("Phone", -50)
except ValueError as e:
    print("Error:", e)

print("The program continues")
```

Common built-in errors

`ValueError` – wrong value · `TypeError` – wrong type · `KeyError` – missing key

Output

```
Error: Negative price
The program continues
```

Better a loud error now than a silent wrong value later.

Private helper methods

Methods that are only meant to be used inside the class

order.py

```python
class Order:
    def __init__(self, items):
        self.items = items   # [(name, price), ...]

    def __subtotal(self):          # private helper
        return sum(price for name, price in self.items)

    def __shipping(self):          # private helper
        return 0 if self.__subtotal() >= 50 else 7.5

    def total(self):               # public method
        return self.__subtotal() + self.__shipping()

o = Order([("Book", 30), ("Pen", 5)])
print(o.total())
```

Output

```
42.5
```



When to use them

- Break a long method into small steps
- Hide details the user of the class does not need
- You can change them later without breaking other code

In-class task 2: a safe Product class

15 min

Work in pairs · 15 minutes



Write a class Product

- Attributes `name`, private `__price`, private `__stock`
- `price` as a `@property`; the setter raises `ValueError` for negative prices
- `sell(quantity)` lowers the stock, raises `ValueError` if not enough stock
- `apply_discount(percent)` only accepts 0–50 %
- `__str__` → `Mouse: 18.0 CHF (9 in stock)`

test it like this

```
m = Product("Mouse", 20.0, 10)
m.sell(1)
m.apply_discount(10)
print(m)
m.sell(100)
```

Expected output

```
Mouse: 18.0 CHF (9 in stock)
ValueError: Not enough stock
```

03

Class-level data

Data shared by all objects of a class

Instance variables vs class variables

Some data belongs to each object, some to the whole class

class_variables.py

```python
class Student:
    university = "Uni Liechtenstein"   # class var
    count = 0                          # class var

    def __init__(self, name):
        self.name = name               # instance var
        Student.count += 1

a = Student("Anna")
b = Student("Ben")
print(a.name, "|", b.name)
print(a.university)
print(Student.count)
```

Output

```
Anna | Ben
Uni Liechtenstein
2
```

- **Instance** variable: one per object (`self.name`)
- **Class** variable: one shared copy (`Student.count`)

Class methods and static methods

Methods that do not need a specific object

methods.py

```python
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    @classmethod
    def from_text(cls, text):      # cls = the class
        name, age = text.split(",")
        return cls(name, int(age))  # new Student

    @staticmethod
    def is_valid_age(age):         # no self/cls
        return 16 <= age <= 99

s = Student.from_text("Chen,20")
print(s.name, s.age)
print(Student.is_valid_age(150))
```

Output

```
Chen 20
False
```

- `@classmethod` gets the class as `cls`: great for **alternative constructors**
- `@staticmethod` is a normal helper function that lives inside the class

04

Inheritance & polymorphism

Reuse code and treat objects alike

Classes with similarities

Copy-paste code is a warning sign

student.py

```python
class Student:
    def __init__(self, name, email, sid):
        self.name = name
        self.email = email
        self.sid = sid

    def contact(self):
        return f"{self.name} <{self.email}>"
```

teacher.py

```python
class Teacher:
    def __init__(self, name, email, office):
        self.name = name
        self.email = email
        self.office = office

    def contact(self):
        return f"{self.name} <{self.email}>"
```

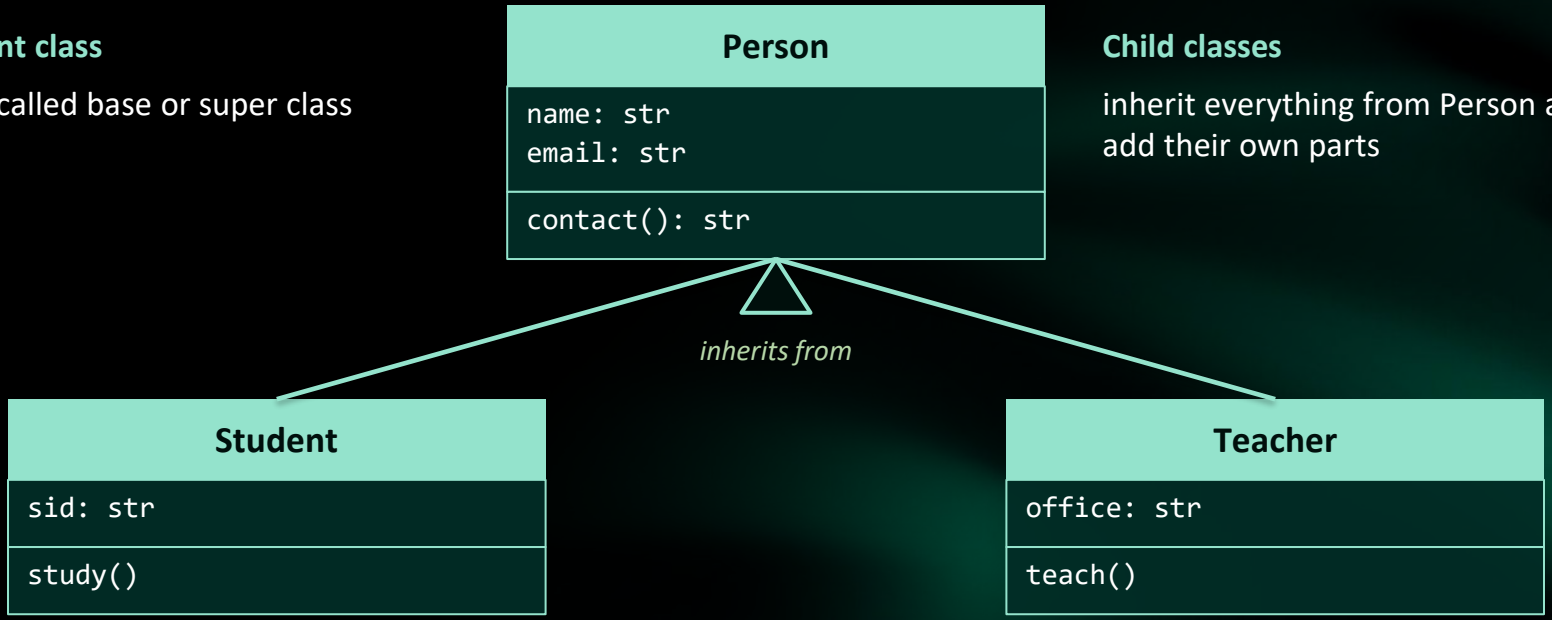
The highlighted lines are identical. If we change how `contact()` works, we must change it in **every** class. Inheritance solves this.

Inheritance: the is-a relationship

A Student is a Person. A Teacher is a Person.

Parent class

also called base or super class



Child classes

inherit everything from Person and add their own parts

Inheritance in Python: class Child(Parent)

super().__init__(...) lets the parent set up its part of the object

inheritance.py

```python
class Person:
    def __init__(self, name, email):
        self.name = name
        self.email = email

    def contact(self):
        return f"{self.name} <{self.email}>"

class Student(Person):                    # inherits
    def __init__(self, name, email, sid):
        super().__init__(name, email)       # parent
        self.sid = sid                      # new part

anna = Student("Anna", "anna@uni.li", "S-102")
print(anna.contact())       # inherited method
print(anna.sid)
```

Output

```
Anna <anna@uni.li>
S-102
```



Key points

- Write the parent in brackets
- Call `super().__init__()`
- The child gets all parent methods for free

Overriding: a child can change inherited behaviour

Same method name in the child class = the child's version wins

override.py

```
class Teacher(Person):
    def __init__(self, name, email, office):
        super().__init__(name, email)
        self.office = office

    def contact(self):                          # override
        base = super().contact()                # reuse parent
        return base + f", office {self.office}"

p = Person("Max", "max@uni.li")
t = Teacher("Dr. Ebrahimi", "pejman.ebrahimi@uni.li", "S2")
print(p.contact())
print(t.contact())
```



Two options

- **Override**: write the method again in the child
- **Extend**: call `super().method()` and add to it

Output

```
Max <max@uni.li>
Dr. Ebrahimi <pejman.ebrahimi@uni.li>, office S2
```

Checking types: isinstance()

A child object is also an instance of its parent class

isinstance.py

```
anna = Student("Anna", "anna@uni.li", "S-102")
max_ = Person("Max", "max@uni.li")

print(isinstance(anna, Student))
print(isinstance(anna, Person))   # is-a!
print(isinstance(max_, Student))  # no
print(issubclass(Student, Person))
```

Output

```
True
True
False
True
```

Is-a test: “A Student **is a** Person”

Every class in Python silently inherits from `object`. That is where default behaviour such as the `<__main__.Student object at 0x…>` text comes from.

Polymorphism: same call, different behaviour

“Many forms”: each object answers area() in its own way

shapes.py

```python
import math

class Shape:
    def area(self):
        return 0

class Circle(Shape):
    def __init__(self, r):
        self.r = r
    def area(self):
        return math.pi * self.r ** 2

class Square(Shape):
    def __init__(self, side):
        self.side = side
    def area(self):
        return self.side ** 2
```

use them all the same way

```python
shapes = [Circle(1), Square(3),
          Circle(2.5)]

for sh in shapes:
    name = type(sh).__name__
    print(name, round(sh.area(), 2))
```

Output

```
Circle 3.14
Square 9
Circle 19.63
```

The loop does not care which shape it is. Add a `Triangle` later and the loop still works.

Inheritance and private attributes

__private stays inside its own class; _protected is shared with children

access.py

```
class Account:
    def __init__(self, balance):
        self.__balance = balance    # private
        self._owner = "Anna"        # protected

class Savings(Account):
    def show(self):
        print(self._owner)          # works
        print(self.__balance)       # fails!

Savings(100).show()
```



Rule of thumb

- Use __x for data only this class may touch
- Use _x for data child classes may use
- Or give children a getter / property

Output

```
Anna
Traceback (most recent call last):
AttributeError: 'Savings' object has no attribute '_Savings__balance'
```

Special methods make objects feel built-in

Methods with double underscores are called by Python itself

money.py

```
class Money:
    def __init__(self, amount):
        self.amount = amount
    def __str__(self):              # print(m)
        return f"{self.amount:.2f} CHF"
    def __eq__(self, other):        # m1 == m2
        return self.amount == other.amount
    def __lt__(self, other):        # m1 < m2, sorted()
        return self.amount < other.amount
    def __add__(self, other):       # m1 + m2
        return Money(self.amount + other.amount)

a, b = Money(5), Money(12.5)
print(a + b, a == Money(5), a < b)
print(min([b, a]))
```

Output

```
17.50 CHF True True
5.00 CHF
```



Most useful

- `__init__` · create
- `__str__` · print
- `__eq__` · compare ==
- `__lt__` · sort with <
- `__len__` · len(obj)

Is-a or has-a? Choosing the right relationship

A key decision when you design a system

Inheritance (is-a)

A `Dog` **is an** `Animal`.

→ `class Dog(Animal)`

Composition (has-a)

A `Car` **has an** `Engine`.

→ `self.engine = Engine()`

Quick quiz — is-a or has-a?

Manager – Employee

Library – Book

Circle – Shape

Course – Student

Tip: when in doubt, prefer **composition**. Use inheritance only when “is a” is really true.

In-class task 3: vehicles

15 min

Work in pairs · 15 minutes

Your task

- Base class `Vehicle` with `brand` and `wheels`, method `describe()` and `fare(km)` returning `0`
- `Taxi(Vehicle)` – 4 wheels, fare = 5 + 2.5 per km
- `Bike(Vehicle)` – 2 wheels, fare = 1 per started 10 min (assume 4 min per km)
- Put several vehicles in a list and print `describe()` and the fare for 6 km
- Draw the UML diagram with the inheritance arrows

test it like this

```
fleet = [Taxi("Toyota"), Bike("Swap")]
for v in fleet:
    print(v.describe(), "->", v.fare(6))
```

Expected output

```
Toyota with 4 wheels -> 20.0
Swap with 2 wheels -> 3
```

The four pillars of object-oriented programming

Everything from Part 1 and Part 2 in one picture



Encapsulation

Hide the data, allow access only through methods that check the rules



Abstraction

Users of a class only need to know what it does, not how it does it



Inheritance

Child classes reuse and extend a parent class (is-a)



Polymorphism

One method call, different behaviour depending on the object

Summary: Part 2 cheat sheet

cheat_sheet.py

```python
class Person:
    count = 0                              # class variable
    def __init__(self, name):
        self.__name = name                 # private
        Person.count += 1
    @property
    def name(self):                        # getter
        return self.__name
    def describe(self):
        return f"I am {self.name}"

class Student(Person):                     # inheritance
    def __init__(self, name, course):
        super().__init__(name)
        self.course = course               # has-a (Course)
    def describe(self):                    # override
        return super().describe() + " and I study"
```

Key takeaways

- Objects can hold other objects (has-a)
- `__private` + methods / `@property` protect data
- `raise ValueError` for invalid input
- `class Child(Parent)` + `super()` (is-a)
- Polymorphism: same call, different behaviour

Homework 2: a mini library system

Submit one .py file and a UML class diagram (PDF or photo) on Moodle

Requirements

- `LibraryItem` (base): `title`, `year`, private `__available = True`; methods `checkout()`, `give_back()` and `is_available()`
- `checkout()` raises `ValueError` if already borrowed
- `Book(LibraryItem)` adds `author`; `DVD(LibraryItem)` adds `minutes`
- Both override `__str__`
- `Library` **has** a list of items: `add()`, `available_items()`, `find(title)` → item or `None`

example use

```
lib = Library()
lib.add(Book("Dune", 1965, "F. Herbert"))
lib.add(DVD("Inception", 2010, 148))
lib.find("Dune").checkout()
for item in lib.available_items():
    print(item)
lib.find("Dune").checkout()
```

Output

```
DVD: Inception, 148 min (2010)
Traceback (most recent call last):
ValueError: 'Dune' is already borrowed
```

Thank you!

Dr. Pejman Ebrahimi

Do you have any questions?

 pejman.ebrahimi@uni.li